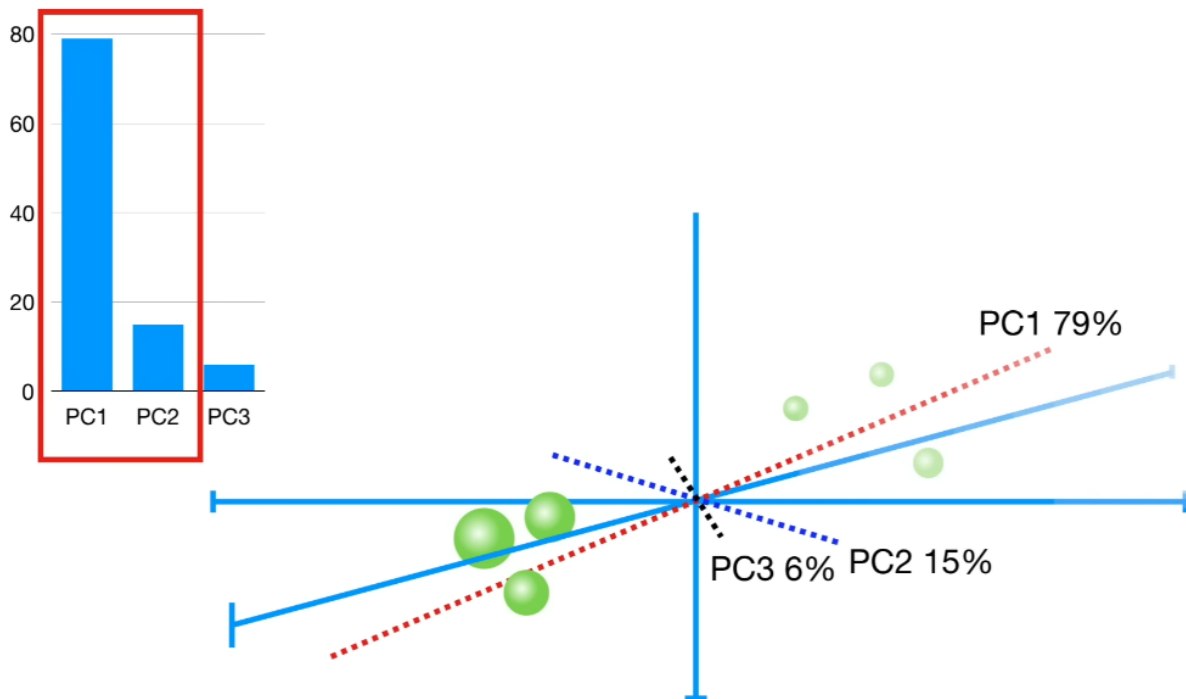


ЗАДАЧА ПОНИЖЕНИЯ РАЗМЕРНОСТИ



План:

1. Введение в снижение размерности. Задача понижения размерности.
2. Методы выбора признаков (Feature Selection).
3. Линейные методы понижения размерности. Метод главных компонент (Principal Component Analysis, PCA).
4. Нелинейные методы понижения размерности. Ядровый анализ главных компонент (kernel PCA). UMAP.
5. Нелинейные методы понижения размерности: t-SNE, Isomap.



Машинное обучение

КОНСТРУИРОВАНИЕ ПРИЗНАКОВ

ПРИНЦИПЫ И ТЕХНИКИ ДЛЯ АНАЛИТИКОВ

Предисловие

Введение

Глава 1. Процесс машинного обучения

Глава 2. Забавные трюки с простыми числами

Глава 3. Текстовые данные: выравнивание, фильтрация и разбиение

Глава 4. Эффекты масштабирования признаков: от множества слов к TF-IDF

Глава 5. Категориальные переменные: подсчет яиц в эпоху роботизированных цыплят

Глава 6. Уменьшение размерности: сжатие данных в блин с помощью PCA

Глава 7. Нелинейное извлечение признаков с помощью стекирования моделей методом k-средних

Глава 8. Автоматизация извлечения признаков: извлечение признаков изображения и глубокое обучение

Глава 9. Назад в признаки: рекомендации научных статей

Приложение А. Основы линейного моделирования и линейной алгебры

Методы выбора признаков условно делят на 3 группы:

- ☐ методы фильтрации (Filter Methods),
- ☐ встроенные методы (Embedded Methods),
- ☐ методы-оболочки (Wrapper Methods).

**Выбор подходящего метода выбора признаков
не всегда очевиден и зависит
от задачи и имеющихся данных!**

МЕТОДЫ ФИЛЬТРАЦИИ (FILTER METHODS)

МЕТОДЫ ФИЛЬТРАЦИИ

Методы фильтрации отбирают признаки на основе их статистических свойств **до обучения модели ML**.

К ним можно отнести следующее:

- ❑ предварительный анализ данных (например, удаление признака, у которого большинство значений пропущено);
- ❑ оценка признаков с помощью некоторого статистического критерия (корреляция и др.);
- ❑ экспертная оценка (удаление признаков, которые не подходят по смыслу, или признаков с некорректными значениями).

МЕТОДЫ ФИЛЬТРАЦИИ: ОДНОМЕРНЫЙ ОТБОР ПРИЗНАКОВ

Методы фильтрации основаны на статистических методах и, как правило, рассматривают каждый признак независимо; позволяют оценить и ранжировать признаки по значимости, за которую принимается степень корреляции этого признака с целевой переменной.

Одномерный выбор признаков работает путем выбора лучших признаков на основе одномерных статистических тестов. Это можно рассматривать как этап предварительной обработки.

Одномерный отбор признаков предполагает, что для каждого признака оценивается **степень его влияния на целевую переменную**. Признаки с небольшим влиянием удаляются.

МЕТОДЫ ФИЛЬТРАЦИИ

❑ Корреляция.

- Удаление признаков с высокой корреляцией (>0.9) для уменьшения многоколлинеарности.
- Использование статистических мер для исследования взаимосвязей в данных:
 - ✓ коэффициент ранговой корреляции Спирмена (Spearman) — для оценки монотонной зависимости;
 - ✓ критерий Хи-квадрат (Chi-squared test) — для проверки взаимосвязи между категориальными переменными;
 - ✓ коэффициент корреляции Пирсона (Pearson) — для оценки линейной зависимости между количественными переменными.

МЕТОДЫ ФИЛЬТРАЦИИ

□ Дисперсия.

- Удаление константных и почти константных признаков (**VarianceThreshold**) — это эффективный способ избавиться от признаков, которые не вносят полезной информации.

□ Статистические тесты на значимость.

- ANOVA (**A**nalysis of **V**ariance, анализ дисперсии) — используется для сравнения средних значений между несколькими группами.,
- F-test (F-критерий) — применяется для проверки гипотез о равенстве дисперсий между группами.

МЕТОДЫ ФИЛЬТРАЦИИ

□ Взаимная информация (Mutual Information, MI).

В контексте понижения размерности, MI является важной концепцией, которая помогает **оценить зависимость между переменными и сохранять информацию при преобразовании данных в пространство меньшей размерности (в низкоразмерное пространство).**

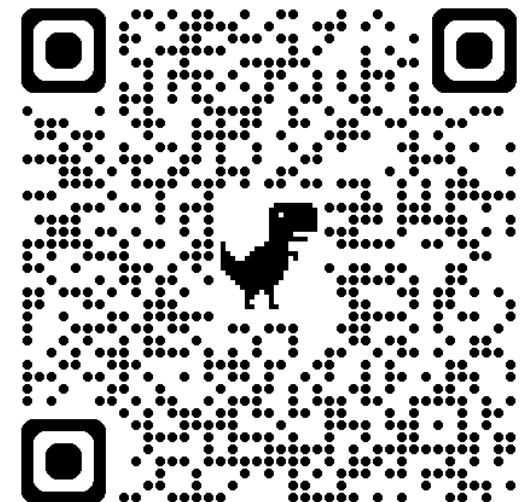
Использование MI может помочь как в **предварительном анализе данных**, так и в **выборе подходящих методов** для эффективного представления данных в низкоразмерном пространстве.

χ^2 (chi2)

Тест χ^2 используется в задачах классификации.

- Оценивает зависимость между признаками и классами целевой переменной.
- Требует неотрицательных и правильно отмасштабированных признаков.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.chi2.html



```
from sklearn.feature_selection import f_classif
```

```
from sklearn.feature_selection import f_regression
```

F-тест

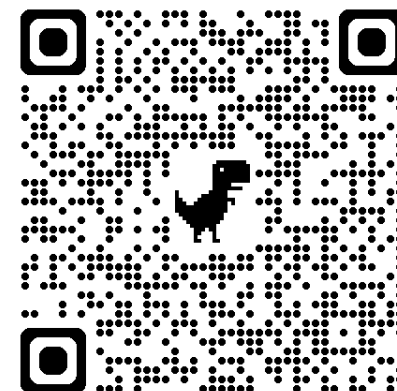
Оценивает степень **линейной зависимости** между **предикторами** и **целевой переменной**, поэтому он лучше всего подойдёт для линейных моделей.

Реализован в Scikit-learn как

f_regression задача регрессии

f_classif задача классификации

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.f_regression.html

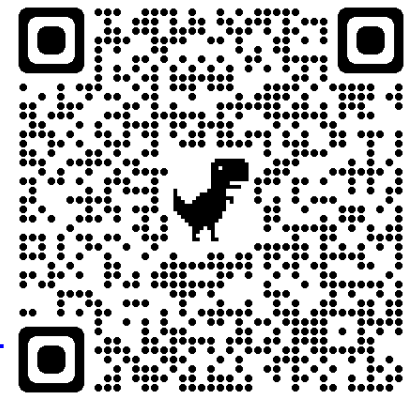


```
from sklearn.feature_selection import SelectKBest
```



SelectKBest оценивает взаимосвязь предикторов с целевой переменной используя **статистические тесты**, позволяя отобрать соответственно заданное количество и долю наилучших по заданному критерию признаков.

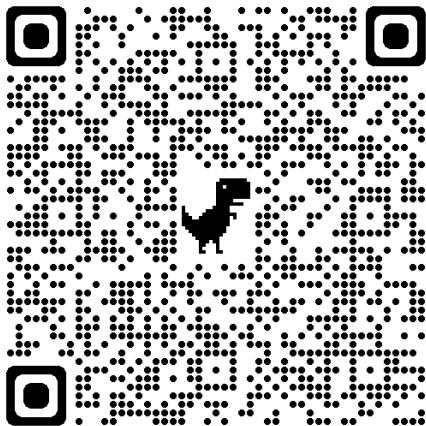
https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html



```
from sklearn.feature_selection import VarianceThreshold
```

VarianceThreshold удаляет все признаки с низкой дисперсией. Этот алгоритм выбора признаков рассматривает только признаки (X), а не желаемые результаты (y) и, таким образом, может использоваться для обучения без учителя.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html#sklearn.feature_selection.VarianceThreshold



Отбор признаков с высокой дисперсией

В общем отбор признаков предполагает, что из набора данных исключают те признаки, которые практически не несут в себе достаточно информации об объектах.

Самый очевидный способ отбора признаков — брать признаки (предикторы) с высокой дисперсией.

```
X.describe()
```

	Customer_Age	Dependent_count	Education_Level	Income_Category	Card_Category	Months_on_book	Total_Relationship_Count
count	3254.000000	3254.000000	3254.000000	3254.000000	3254.000000	3254.000000	3254.000000
mean	46.766749	2.413337	2.889367	1.890596	0.076521	36.280578	3.750461
std	7.371566	1.285434	1.990770	1.417368	0.330788	7.530996	1.550401
min	26.000000	0.000000	0.000000	0.000000	0.000000	13.000000	1.000000
25%	42.000000	1.250000	1.000000	1.000000	0.000000	32.000000	3.000000
50%	47.000000	2.000000	3.000000	1.000000	0.000000	36.000000	4.000000
75%	52.000000	3.000000	5.000000	3.000000	0.000000	40.000000	5.000000
max	68.000000	5.000000	6.000000	5.000000	3.000000	56.000000	6.000000

Взаимная информация (Mutual Information)

Взаимная информация показывает насколько чётко определена целевая переменная если известны значения предиктора. Этот тип тестов считается самым удобным в использовании - он хорошо работает "из коробки" и позволяет находить нелинейные зависимости.

Реализован в Sklearn как

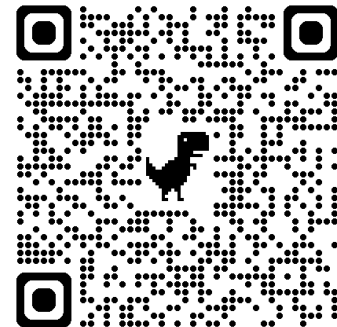
`mutual_info_regression`

задача регрессии

`mutual_info_classif`

задача классификации

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.mutual_info_classif.html



from sklearn.feature_selection import *



Задачу выбора признаков в исходных данных можно автоматизировать с помощью специального модуля **feature_selection** в **Sklearn**, который отбирает признаки по следующим параметрам:

- ❑ высокий процент **пропущенных значений**;
- ❑ **коллинеарность** (удаляются признаки, у которых коэффициент корреляции больше порогового) — метод **identify_collinear**;
- ❑ **вариативность** (удаляются признаки, состоящие из одного значения или имеют очень малый диапазон значений);
- ❑ признаки с **низкой значимостью**: функция **identify_low_importance**;
- ❑ признаки с **нулевой важностью**: функция **identify_zero_importance**.

Выбор признаков обычно используется как **предварительный этап обработки** перед выполнением обучения модели. Рекомендуемый способ сделать это в Sklearn:

использовать **Pipeline** (конвейер обработки данных)

1.13.1. Removing features with low variance

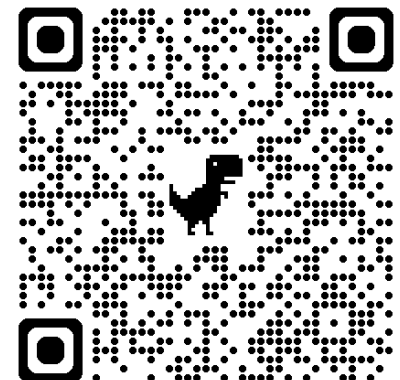
1.13.2. Univariate feature selection

1.13.3. Recursive feature elimination

1.13.4. Feature selection using SelectFromModel

1.13.5. Sequential Feature Selection

1.13.6. Feature selection as part of a pipeline



```
clf = Pipeline([
    ('feature_selection', SelectFromModel(LinearSVC(penalty="l1"))),
    ('classification', RandomForestClassifier())
])
clf.fit(X, y)
```

Pipeline состоит из двух этапов:

1. Выбор признаков: используется `SelectFromModel` в сочетании с `LinearSVC` с L1-регуляризацией для оценки важности признаков.

2. Классификация: применяется `RandomForestClassifier`, который обучается на выходных данных после выбора признаков.

Это означает, что классификатор использует только те признаки, которые были отобраны на предыдущем этапе, что может повысить его производительность и снизить вероятность переобучения.

МЕТОДЫ ФИЛЬТРАЦИИ

Плюсы:

- ☐ Быстрые и масштабируемые.
- ☐ Не зависят от модели.

Минусы:

- ☐ Игнорируют взаимодействие признаков.

Когда использовать?

- ☐ Первичный отбор признаков.
- ☐ Большие датасеты.

ВСТРОЕННЫЕ МЕТОДЫ (EMBEDDED METHODS)

EMBEDDED METHODS

Принцип: Встроенные методы выполняют **отбор признаков во время обучения модели ML**, оптимизируя их набор для достижения лучшей точности. К этим методам можно отнести

- ❑ **регуляризацию в линейных моделях (обычно L1);**
- ❑ **расчёт важности признаков в алгоритмах с решающими деревьями.**

Эти алгоритмы требуют **меньше вычислений, чем wrapper methods** (но **больше, чем методы фильтрации**).

Основной метод из этой категории:
регуляризация.

Идея регуляризации: построить алгоритм минимизирующий **не только ошибку, но и количество используемых переменных.**

Обученные регрессионные модели и деревья решений хранят информацию о значимости признаков (коэффициенты в линейных моделях и значения значимости признаков у деревьев). Можно отсортировать признаки по этим значениям и производить отбор

EMBEDDED METHODS

L1-регуляризация (Lasso):

Нулевые коэффициенты = неважные признаки.

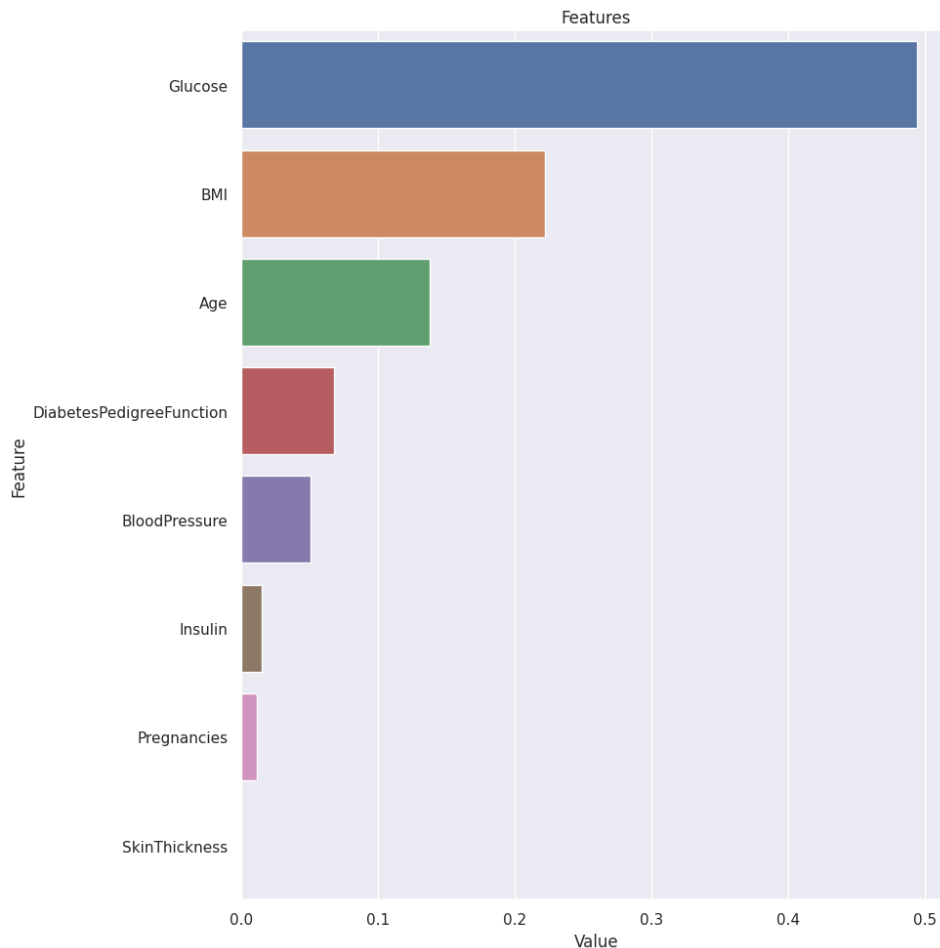
```
from sklearn.linear_model import LassoCV  
model = LassoCV().fit(X, y)  
selected_features = X.columns[model.coef_ != 0]
```


EMBEDDED METHODS

Взять обученное дерево и выделить нужное количество признаков с самой высокой значимостью.

Деревья решений:

Важность признаков через **feature_importances_**.



EMBEDDED METHODS

Плюсы:

Баланс скорости и точности.

Учитывают взаимодействие признаков.

Минусы:

Зависят от выбора модели.

Когда использовать?

- ☐ Когда модель уже выбрана (например, Lasso для линейных задач).
- ☐ Для интерпретируемости (например, важность признаков в Random Forest).

МЕТОДЫ-ОБОЛОЧКИ (WRAPPER METHODS)

ПРИНЦИП: Итеративно выбирают подмножества признаков, оценивая их полезность помощью **производительности модели ML**.

- ❑ Все wrapper methods **требуют гораздо больших вычислений**, чем методы фильтрации. Методы оболочки используют модель ML и обучают ее, чтобы решить, важна ли она для окончательной модели или нет.
- ❑ В случае большого количества признаков и небольшого размера тренировочного сета эти методы **имеют опасность переобучения**.

```
from sklearn.feature_selection import RFE
```



Recursive Feature Elimination (RFE):

Рекурсивно удаляет наименее важные признаки.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.RFE.html



Рекурсивный отбор признаков

Рекурсивный отбор признаков предполагает обучение модели. Сначала модель обучается на всех признаках, а затем следует **исключить из модели признаки с низкой значимостью**. Делать это до тех пор, пока не останется столько признаков, сколько нужно получить

Важный момент: каким образом исключаются признаки; это происходит на основе значения значимости признака.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression
data= load_data()
X = data["data"]
Y = data["target"]
lr = LinearRegression()
rfe = RFE(lr, 5) #выбор 5 наиболее информативных признаков
selector = rfe.fit(X,Y)
```

Пример (RFE с логистической регрессией):

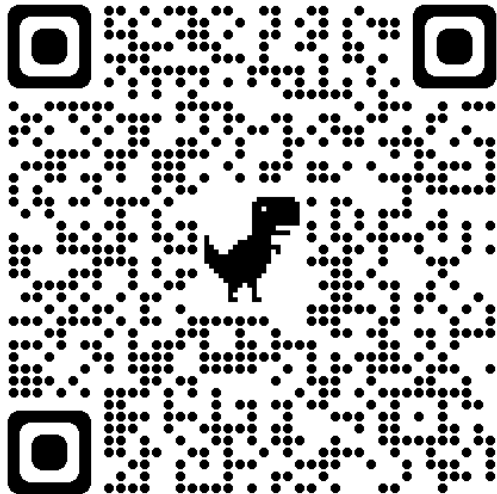
```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
rfe = RFE(estimator=model, n_features_to_select=5)
X_new = rfe.fit_transform(X, y)
```

Sequential Feature Selection:

Forward Selection — добавляет признаки по одному.

Backward Elimination — удаляет признаки по одному.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SequentialFeatureSelector.html



Плюсы:

- Учитывают взаимодействие признаков.
- Часто дают лучшую точность.

Минусы:

- Вычислительно дорогие.
- Риск переобучения.

Когда использовать?

- Средние датасеты.
- Критична точность модели.

Feature Selection vs. Снижение размерности		
Критерий	Feature Selection	Снижение размерности (PCA, t-SNE)
Сохраняет признаки	Да (исходные колонки)	Нет (новые компоненты)
Интерпретируемость	Высокая	Низкая
Скорость	Зависит от метода	Быстро (кроме t-SNE)
Использование	Когда важны имена признаков	Для визуализации/сжатия данных

Feature Selection **vs.** Снижение размерности

Когда выбирать feature selection?

- ☐ Нужно объяснить вклад признаков (например, медицинская диагностика).
- ☐ Признаки имеют физический смысл (цена, возраст, генетические маркеры).

Когда выбирать PCA?

- ☐ Мультиколлинеарность признаков.
- ☐ Данные для визуализации.

Практические рекомендации

- ❑ **Начинайте с методов фильтрации** для быстрого отсева неинформативных признаков.
- ❑ **Для финальной модели** используйте обертки или встроенные методы.
- ❑ **Проверяйте стабильность:** запускайте отбор несколько раз с разными значениями параметра `random_state`.
- ❑ **Избегайте утечки данных (data leak):** применяйте методы `feature selection` только на тренировочной выборке (train).

Выводы

Выбор подходов зависит от задачи:

- Нужна интерпретация → **Feature Selection**.
- Нужно сжать данные → **PCA/UMAP/t-SNE**.

МЕТОДЫ СНИЖЕНИЯ РАЗМЕРНОСТИ (DIMENSIONALITY REDUCTION) И ВЫДЕЛЕНИЕ ПРИЗНАКОВ (FEATURE EXTRACTION)

Методы снижения размерности и методы извлечения признаков не являются одним и тем же, хотя они **тесно связаны** и **часто пересекаются**.

МЕТОДЫ СНИЖЕНИЯ РАЗМЕРНОСТИ ДАННЫХ

Основные методы снижения размерности данных:

- ❑ анализ главных компонент (РСА);**
- ❑ канонический корреляционный анализ (ССА),
факторный анализ (ФА);
- ❑ нелинейные методы снижения размерности.**

Методы выделения признаков (Feature Extraction) каким-то образом **составляют из исходных признаков новые признаки**, все также полностью описывающие пространство набора данных, но уменьшая его размерность и теряя в репрезентативности данных, т.к. становится непонятно, за что отвечают новые признаки.

Ключевое сходство

1. Снижение размерности (Dimensionality Reduction)

Цель: Уменьшить количество признаков в данных, сохраняя как можно больше полезной информации.

Методы:

- ❑ **Линейные:** PCA (Principal Component Analysis), LDA (Linear Discriminant Analysis), SVD (Singular Value Decomposition).
- ❑ **Не линейные:** t-SNE (t-Distributed Stochastic Neighbor Embedding), UMAP (Uniform Manifold Approximation and Projection), Autoencoders.

Применение: Ускорение обучения моделей, борьба с “проклятием размерности”, визуализация данных.

2. Извлечение признаков (Feature Extraction)

Цель: Преобразовать исходные признаки в новые, более информативные или интерпретируемые.

Методы:

- ❑ PCA (можно считать и методом Feature Extraction, т.к. он создаёт **новые признаки — главные компоненты**).
- ❑ Автокодировщики (Autoencoders).
- ❑ Методы, основанные на преобразованиях (например, вейвлет-преобразования для изображений или текстов).

Применение: Улучшение качества моделей, выделение скрытых закономерностей.

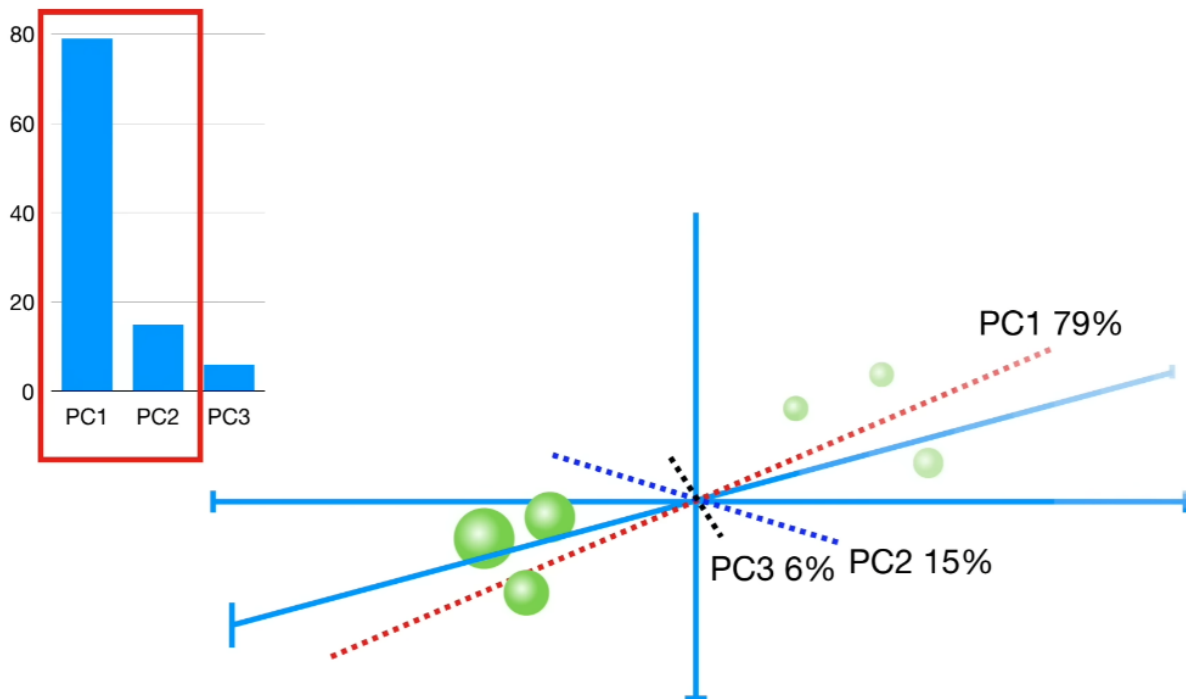
Ключевые различия

Feature Extraction может не обязательно уменьшать размерность, а просто создавать новые признаки (например, извлечение гистограмм ориентированных градиентов (HOG) в компьютерном зрении).

Dimensionality Reduction всегда уменьшает число признаков, но иногда делает это через Feature Extraction (как PCA).

Не все методы Feature Extraction снижают размерность, и не все методы Dimensionality Reduction создают новые признаки. Однако многие алгоритмы (например, PCA) относятся к обоим категориям.

МЕТОД ГЛАВНЫХ КОМПОНЕНТ (Principal Components Analysis, PCA)



Метод главных компонент (РСА)

МЕТОД ГЛАВНЫХ КОМПОНЕНТ — один из основных способов в ML уменьшить размерность данных, потеряв наименьшее количество информации.

ГЛАВНЫЕ КОМПОНЕНТЫ — новые переменные, с помощью которых будут описывать объекты.

1901 г. РСА изобретен **К. Пирсоном** (Karl Pearson)

РСА применяется во многих областях, таких как распознавание образов, компьютерное зрение, сжатие данных, NLP и т.п.

Иногда РСА называют **преобразованием Карунена-Лоэва** (англ. *Karhunen-Loeve*) или **преобразованием Хотеллинга** (англ. *Hotelling transform*).

ИДЕЯ PCA

Идея PCA: **уменьшить** количество переменных набора данных, **сохранив** при этом как можно **больше информации**, т.е. найти новые признаки, называемые **главными компонентами (ГК)**, которые **максимально коррелируют с исходными данными**.

Эти главные компоненты формируют новый базис в пространстве признаков, исключая лишнюю информацию и снижая размерность.

Вычисление ГК сводится к вычислению собственных векторов и собственных значений **ковариационной матрицы** исходных данных.

ПРИНЦИП РАБОТЫ РСА

Например, есть матрица данных X , где каждая строка представляет собой наблюдение, а каждый столбец — признак.

Цель: найти такие **новые признаки (главные компоненты)**, которые наилучшим образом описывают изменчивость данных.

РСА возвращает новую матрицу, в которой столбцы представляют собой линейные комбинации исходных столбцов. **Эти линейные комбинации максимизируют дисперсию.**

Главные компоненты вычисляются как собственные векторы ковариационной матрицы данных.

ГЛАВНАЯ КОМПОНЕНТА

Линейная комбинация предикторных переменных.

НАГРУЗКА (LOADINGS)

Веса, которые позволяют трансформировать предикторы в компоненты.

Синоним: веса.

ГРАФИК КАМЕНИСТОЙ ОСЫПИ (SCREEPLOT)

График дисперсий компонент, показывающий относительную важность компонент

МЕТОД ГЛАВНЫХ КОМПОНЕНТ

1. Стандартизация данных.
2. Вычисление ковариационной матрицы.
3. Вычисление собственных векторов и собственных значений ковариационной матрицы.
4. Сортировка пар **<собственное значение, собственный вектор>** по убыванию. Чем больше число, тем больше дисперсия.
5. Выбор первых **k** пар **<собственное значение, собственный вектор>**, где **k** — размерность целевого пространства.
6. Матрица, составленная из **k** собственных векторов — матрица преобразования из данного пространства в пространство с размерностью **k** . Берем **N** первых собственных векторов, которые соответствуют первым **N** собственным числам. Это и есть **искомые главные компоненты**.
7. Чтобы произвести понижение размерности следует умножить матрицу стандартизированных входных данных (результат п. 1) на матрицу из **k** собственных векторов (результат п. 6).

Основные шаги PCA (1)

1. СТАНДАРТИЗАЦИЯ ДАННЫХ. Прежде чем приступить к вычислению главных компонент, важно **стандартизировать данные**, приводя их к нулевому среднему и единичной дисперсии. Это важно, потому что признаки с разными масштабами могут исказить результаты PCA.

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Основные шаги PCA (2)

2. ВЫЧИСЛЕНИЕ КОВАРИАЦИОННОЙ МАТРИЦЫ.

Цель шага №2: увидеть, есть ли какая-либо связь между переменными во входных данных. Иногда переменные сильно коррелированы (имеют зависимость). Чтобы идентифицировать взаимосвязи, нужно вычислить ковариационную матрицу.

Ковариационная матрица представляет собой симметричную матрицу размера $p \times p$ (где p - количество измерений), элементами которой являются ковариации всех возможных пар исходных переменных.

Например, для 3-мерного набора данных с 3 переменными x , y и z ковариационная матрица представляет собой матрицу 3×3 , состоящую из следующих элементов:

$$\begin{bmatrix} Cov(x, x) & Cov(x, y) & Cov(x, z) \\ Cov(y, x) & Cov(y, y) & Cov(y, z) \\ Cov(z, x) & Cov(z, y) & Cov(z, z) \end{bmatrix}$$

```
cov_matrix = np.cov(X_scaled, rowvar = False)
```

Основные шаги PCA (3)

Что матрица ковариации говорит о связях между переменными?

На самом деле **значение имеет знак ковариации:**

- ❑ если **положительный**, то две переменные увеличиваются или уменьшаются вместе (коррелированы);
- ❑ если **отрицательно**, то одно увеличивается, когда другое уменьшается (обратно коррелированы).

Основные шаги PCA (3)

3. ВЫЧИСЛЕНИЕ СОБСТВЕННЫХ ВЕКТОРОВ И СОБСТВЕННЫХ ЗНАЧЕНИЙ КОВАРИАЦИОННОЙ МАТРИЦЫ. Это можно сделать с использованием различных библиотек, таких как NumPy.

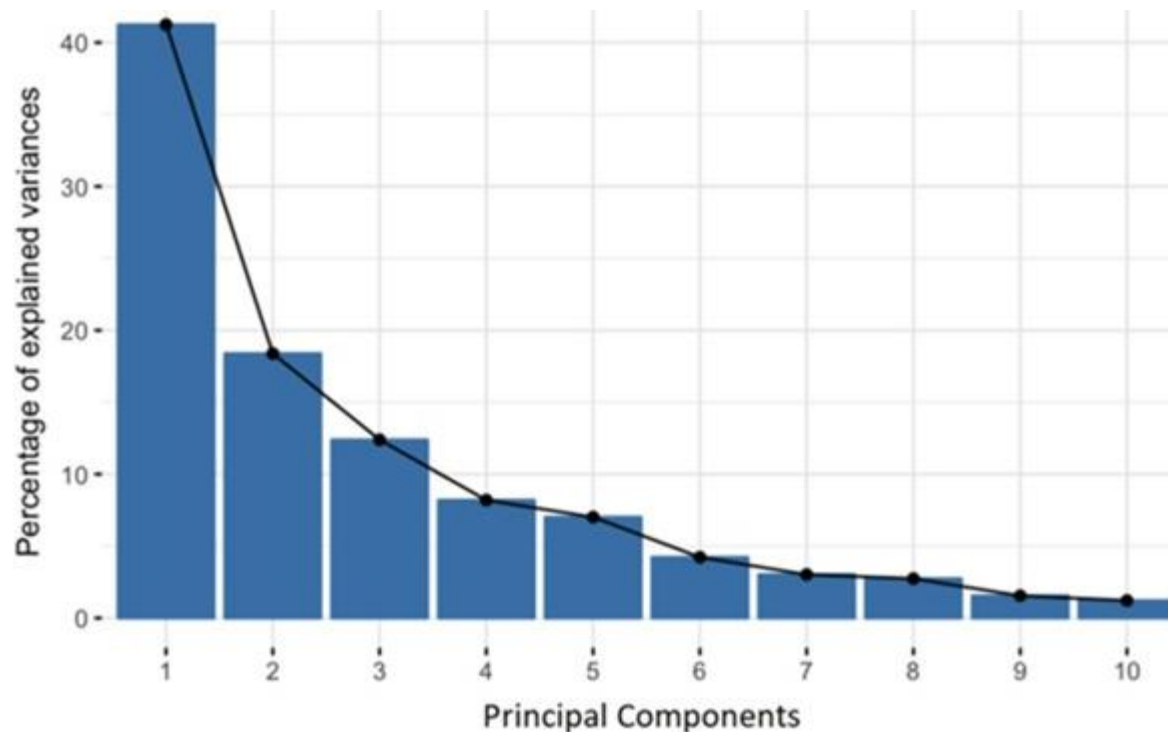
Другое название собственных чисел и собственных векторов – **айгенпары** (**eigenvectors** и **eigenvalues**).

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
```

На самом деле собственные векторы матрицы ковариации являются направлениями осей, где наблюдается наибольшая дисперсия (т.е. раскидана большая часть информации). Эти собственные вектора называют **главными компонентами**. А собственные значения — это коэффициенты, которые отражают величину дисперсии.

Как PCA конструирует главные компоненты? (1)

Идея состоит в том, что 10-мерные данные дают 10 главных компонент, но PCA пытается поместить максимум возможной информации в первый компонент, затем максимум оставшейся информации во второй и т.д., пока не получится следующая диаграмма (рис. 1).



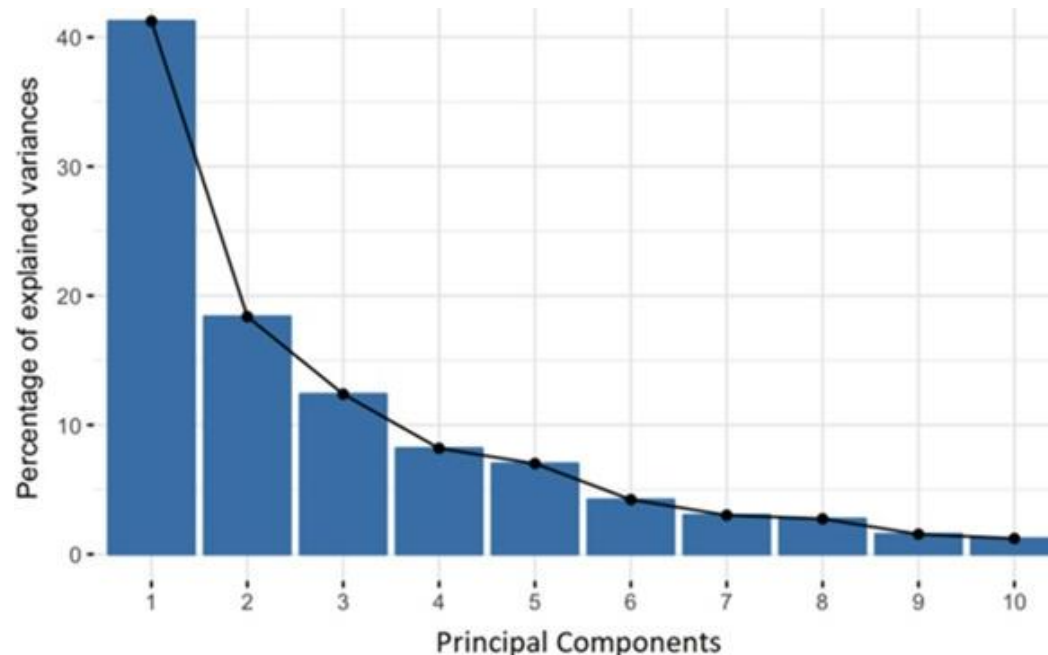
**Отбросив
компоненты с
маленькой инфор-
мацией, можно
уменьшить размер-
ность без особой
потери данных!**

Рис. 1. Количество информации для каждой главной компоненты

Как PCA конструирует главные компоненты? (2)

Поскольку количество главных компонент равно числу изначальных переменных, они строятся таким образом, что первый главный компонент учитывает **максимально возможную дисперсию в наборе данных**.

Например, предположим, что диаграмма рассеяния набора данных выглядит так, как показано ниже (синие точки). Можем ли мы угадать первый главный компонент? Да, это фиолетовая линия, вдоль которой синие точки сильнее всего раскиданы. Или, говоря математически, это линия максимизирует дисперсию.



Как PCA конструирует главные компоненты? (3)

Второй главный компонент рассчитывается таким же образом, при условии, что он не коррелирован первому главному компоненту и учитывает следующую по величине дисперсию. Это продолжается до тех пор, пока не будет вычислено общее количество p главных компонент, равное исходному количеству переменных.

Главные компоненты представляют собой новые признаки, которые можно использовать для анализа или обучения моделей ML.


```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X, y)  
X_pca.shape
```

```
(3254, 2)
```

```
[ ] sum(pca.explained_variance_ratio_)
```

```
0.9924448371305413
```

Видно, что мы сохранили более 99% информации понизив размерность до 2

Выбор числа p главных факторов

- Общая дисперсия данных:

$$Var(X) = \sigma_1^2 + \sigma_2^2 + \dots + \sigma_m^2$$

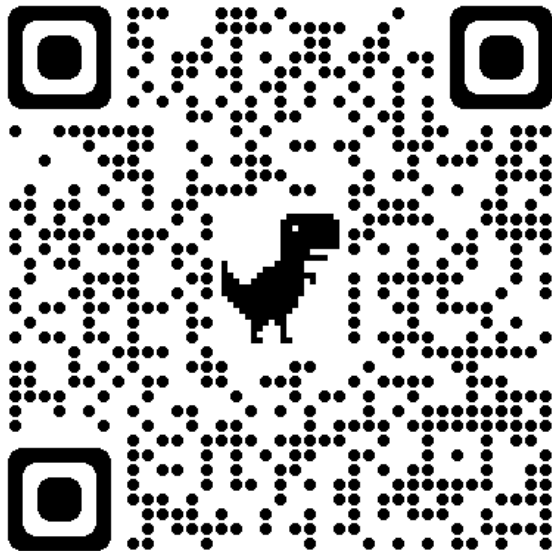
- Доля объясненной дисперсии:

$$\frac{\sigma_1^2 + \sigma_2^2 + \dots + \sigma_k^2}{Var(X)}, p < m$$

- Хорошим значением считается **доля объясненной дисперсии $\geq 80\%$**

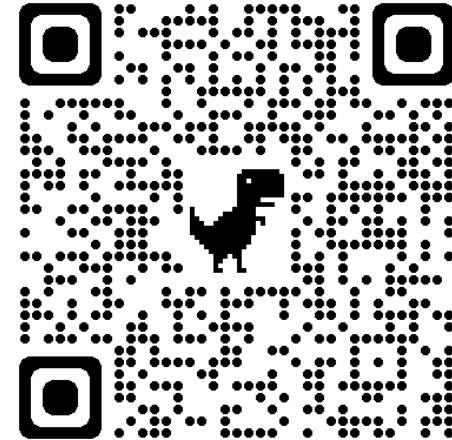
from **sklearn.decomposition** import **PCA**

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>



Как работает метод главных компонент (PCA) на простом примере

<https://habr.com/ru/articles/304214/>



Как применять метод PCA для уменьшения размерности данных

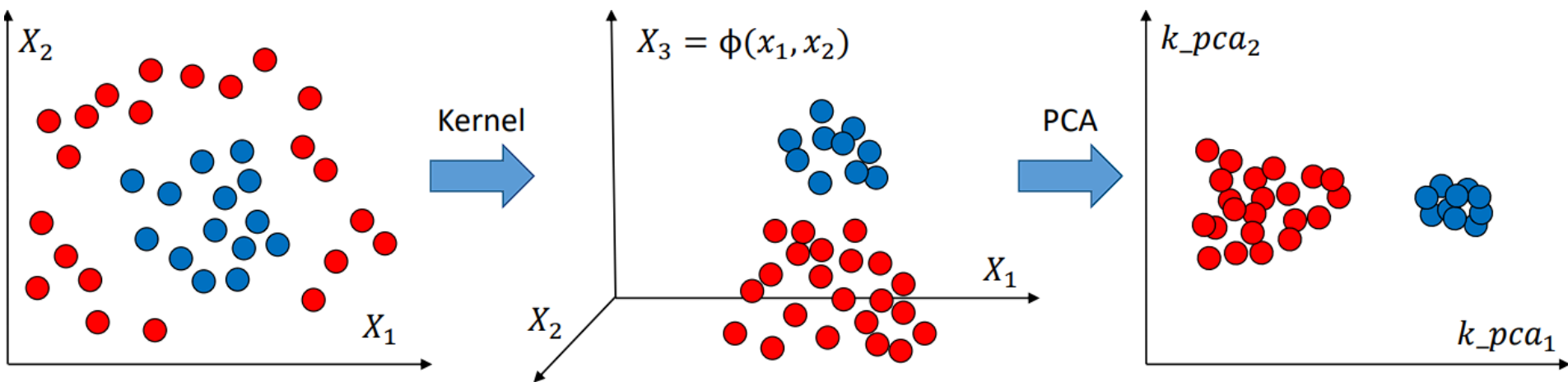
<https://habr.com/ru/companies/otus/articles/769274/>



ЯДРОВЫЙ PCA

Анализ главных компонент (PCA) может быть расширен, чтобы искать **нелинейные “направления” в данных** с использованием ядерных методов. Т. е., вместо поиска линейных комбинаций во входном пространстве, алгоритм ищет линейные комбинации в пространстве признаков, полученном из нелинейного преобразования входного пространства.

ЯДРОВЫЙ PCA



Алгоритм:

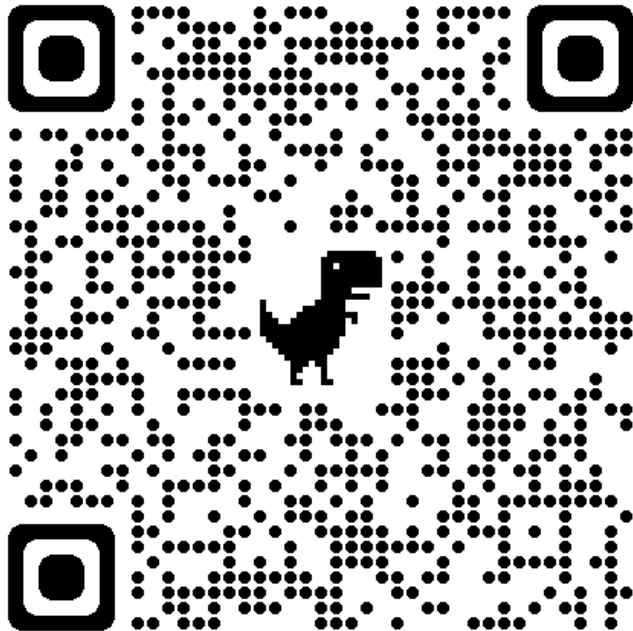
1. Выбор функции ядра и её использование для получения проекции векторов данных в новое пространство большей размерности, где классы становятся линейно разделимыми.
2. Использование обычного метода главных компонент для проецирования полученных векторов в пространство более низкой размерности.

Несколько распространенных ядер:

RBF	$\exp\left(-\frac{\ x - x_j\ ^2}{2\sigma^2}\right)$
Polynomial	$(a \times x^T x_j + b)^d$
Powered	$-\ x - x_j\ ^\beta, 0 < \beta \leq 1$
Log	$-\log\left(1 + \ x - x_j\ ^\beta\right), 0 < \beta \leq 1$
Sigmoid	$\tanh(\sigma \times x^T x_j + r)$

from **sklearn.decomposition** import **KernelPCA**

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.KernelPCA.html>



Плюсы PCA:

- Сокращение количества признаков до наиболее релевантных предикторов.
- Уменьшение размеров облегчает визуализацию данных в двух или трех измерениях.
- Прежде чем обучать другую модель ML с учителем или без учителя, ее можно выполнить в рамках EDA для выявления закономерностей и корреляций с помощью PCA.
- Модели ML быстрее обучаются, имеют тенденцию уменьшать переобучение (избегая проклятия размерности) и их легче интерпретировать, если они снабжены наборами данных меньшей размерности.

Минусы PCA:

- Трудно интерпретировать!

Нелинейные методы понижения признаков

2. Unsupervised learning

2.1. Gaussian mixture models

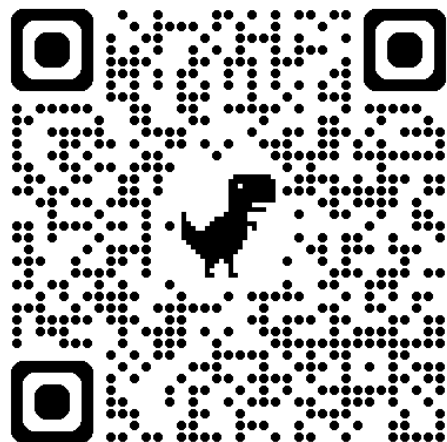
2.2. Manifold learning

2.3. Clustering

2.4. Biclustering

2.5. Decomposing signals in components (matrix factorization problems)

2.6. Covariance estimation



☰ On this page

2.2.1. Introduction

2.2.2. Isomap

2.2.3. Locally Linear Embedding

2.2.4. Modified Locally Linear Embedding

2.2.5. Hessian Eigenmapping

2.2.6. Spectral Embedding

2.2.7. Local Tangent Space Alignment

2.2.8. Multi-dimensional Scaling (MDS)

2.2.9. t-distributed Stochastic Neighbor Embedding (t-SNE)

2.2.10. Tips on practical use

Раздел `sklearn.manifold` в библиотеке Scikit-learn отвечает за **методы снижения размерности и визуализации данных**. Эти методы полезны для работы с высокоразмерными данными, позволяя упростить их представление и выявить скрытые структуры или паттерны.

Эти методы помогают исследовать и визуализировать данные, а также могут быть полезны в предварительной обработке перед применением других алгоритмов ML.

Глобальные структуры vs. локальные структуры

1. Глобальные структуры:

- PCA стремится найти направления, которые охватывают **всю выборку данных**. Это означает, что анализируются все данные одновременно, и результаты отражают общие тенденции и паттерны.

Например, если есть набор данных с несколькими переменными, PCA будет искать **линейные комбинации этих переменных, которые максимизируют общее объясненное разнообразие**. Это может помочь **выявить скрытые факторы или структуры**, которые действуют на уровне **всей выборки**.

Глобальные структуры vs. локальные структуры

2. Локальные структуры:

- В отличие от глобального подхода, локальные структуры фокусируются на **анализе небольших подмножеств данных**. Это может быть полезно для выявления особенностей или паттернов, которые могут существовать только в определенных областях данных.

- Методы, такие как локальное линейное вложение (LLE) или t-SNE, могут быть более подходящими для анализа локальных структур, так как они учитывают близость точек в пространстве.

Глобальные структуры vs. локальные структуры

Почему это важно?

- **Объяснение вариации:** Глобальные структуры помогают понять, какие факторы наиболее влияют на данные в целом. Это **особенно полезно для задач визуализации и уменьшения размерности.**

- **Обнаружение аномалий:** При поиске глобальных структур PCA может помочь выявить аномалии или выбросы, которые могут не быть заметными при локальном анализе.

- **Упрощение данных:** Глобальные структуры позволяют уменьшить размерность данных, сохраняя при этом наиболее важные характеристики, что облегчает дальнейший анализ.

В контексте PCA поиск глобальных структур подразумевает акцент на выявлении общих закономерностей и взаимосвязей в данных, что делает этот метод мощным инструментом **для анализа многомерных наборов данных.**

Метод t-SNE

- ❑ **t-SNE** (англ. **t-Distributed Stochastic Neighbor Embedding** — стохастическое вложение соседей с t-распределением) — метод визуализации данных высокой размерности с помощью представления каждой точки данных в двух или трехмерном пространстве.
- ❑ **Основная идея t-SNE:** конвертировать близость каждой пары точек в исходном пространстве большой размерности в вероятность того, что одна точка данных связана с другой точкой как с ее соседом. Поскольку это **требуется значительных вычислительных ресурсов, сложно использовать эту технику с большим набором данных.**
- ❑ Перед использованием **t-SNE** имеет смысл стандартизировать данные **StandardScaler()**.

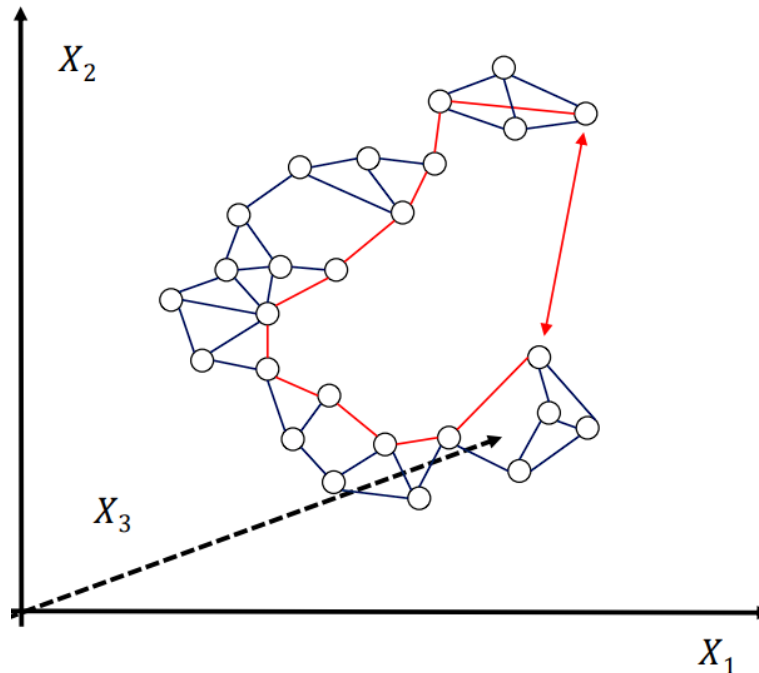
```
from sklearn.manifold import TSNE
```

<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>



Изометрическое картографирование (ISOMap)

- **ISOMap (Isometric Feature Mapping)** — данный алгоритм стремится создать низкоразмерное представление, сохраняющее геодезические расстояния между точками.
- **Геодезическое расстояние** — обобщение расстояния для изогнутых поверхностей.

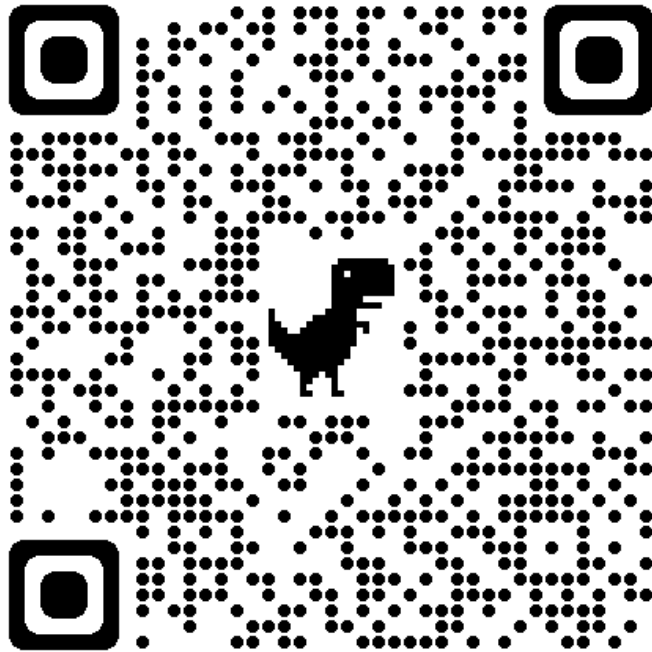


Изометрическое картографирование (ISOMap)

1. Нахождение **k** ближайших соседей для каждого вектора данных.
2. Построение графа, соединяющего ближайших соседей. Вектора, не являющиеся соседями, остаются не связанными.
3. Поиск **геодезического расстояния**: нахождение самого короткого пути (алгоритм Флойда-Уоршелла или Дейкстры) для каждой пары векторов данных.
4. Применение MDS (многомерное шкалирование) для определения новых координат в пространстве низкой размерности.

```
from sklearn.manifold import Isomap
```

<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.Isomap.html#sklearn.manifold.Isomap>



UMAP

- ❑ **UMAP (Uniform Manifold Approximation and Projection)** — это метод для понижения размерности, который позволяет визуализировать и анализировать высокоразмерные данные.
- ❑ UMAP разработан в 2018 г.
- ❑ Эффективен **сохранять глобальные и локальные структуры данных** (по сравнению с другими методами, такими как t-SNE).
- ❑ **Скорость:** UMAP обычно работает быстрее, чем многие другие алгоритмы понижения размерности, особенно на больших наборах данных.

UMAP

- ❑ **Алгоритм:** UMAP основан на теории многообразий и топологии. Он предполагает, что высокоразмерные данные могут быть представлены как многообразие в более высоком пространстве. Алгоритм создает граф, который отражает локальные структуры данных, а затем проецирует его в пространство меньшей размерности.
- ❑ **Параметры:** UMAP имеет несколько ключевых параметров, таких как `n_neighbors` (количество соседей для определения локальной структуры) и `min_dist` (минимальное расстояние между точками в низкоразмерном пространстве), которые позволяют настраивать его поведение в зависимости от специфики данных.

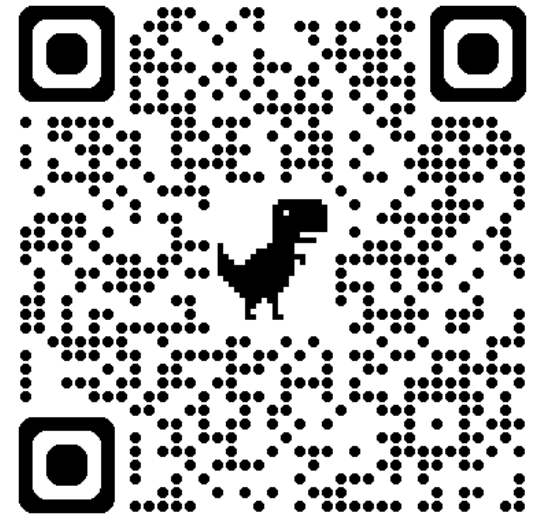
Применение UMAP

- Широко используется в различных областях, включая биоинформатику, обработку изображений, анализ текстов и многое другое.
- Визуализация кластеров в данных, выявления паттернов и подготовки данных для последующего анализа.
- Мощный инструмент для понижения размерности и анализа данных. Его способность сохранять структуру данных и высокая скорость работы делают его предпочтительным выбором в ряде задач анализа данных.



```
import umap # pip install umap-learn
```

<https://umap-learn.readthedocs.io/en/latest/>



End!